



Aprendizaje Autónomo 1

David Brito Toapanta

Facultad de ingenierías digitales y tecnologías emergentes, UIDE.

Lógica de Programación.

Mónica Patricia Salazar Tapia.

28 de mayo de 2026

Instrucciones:**Actividad Autónoma 1 - Selección del Programa a desarrollar / Generación de Diagramas funcionales y Arquitectura de Software.**

A partir del análisis de diferentes alternativas, el estudiante deberá seleccionar un programa a desarrollar. Como primer paso, se realizará el diseño de la solución del problema mediante diagramas de funcionalidad (casos de uso) y de arquitectura del sistema.

Para ello, es fundamental comprender claramente el problema de programación antes de diseñar los diagramas, asegurando que se identifiquen correctamente todas las funcionalidades del sistema.

Opciones de programas a desarrollar

El estudiante deberá escoger una de las siguientes opciones:

- Juego del Ahorcado
- Piedra, papel o tijera
- Juego Atari Pong
- Juego de la serpiente
- Generador seguro de contraseñas
- Juego “Adivina el número” (el computador adivina el número del usuario)

Tipo de actividad: Diseño de diagramas de funcionalidad y arquitectura previo al desarrollo de software.

Tiempo aproximado de desarrollo por el estudiante: 20 horas

Instrucciones para el desarrollo (paso a paso)**1. Investigación:**

Investigar los tipos de diagramas de funcionalidad (casos de uso, diagramas de flujo, etc.) y de arquitectura de software. Seleccionar al menos un tipo de cada uno.

2. Selección del problema:

Elegir el programa a desarrollar y analizar el problema. En esta etapa no se debe programar, sino comprender claramente qué se desea resolver.

3. Diseño funcional:

Diseñar los diagramas de uso que representen todas las funcionalidades del sistema (qué hará el software y cómo interactúa el usuario).

4. Diseño de arquitectura:

Elaborar un diagrama de arquitectura que muestre de forma general cómo estará estructurado el sistema (componentes principales y su relación).

Características de entrega del producto:

El video debe incluir:

- Presentación del software seleccionado
- Explicación del análisis realizado
- Descripción del funcionamiento del sistema basado en los diagramas funcionales
- Explicación general de la arquitectura del sistema

Documento PDF

Debe incluir:

- Diagramas funcionales (casos de uso u otros)
- Diagrama de arquitectura

Introducción:

El presente aprendizaje autónomo 1, tiene como objetivo el diseño previo de un sistema informático que permite comprender de manera clara las funcionalidades que deberá cumplir el software antes de iniciar su programación, en el presente trabajo se realiza el análisis y diseño de un GENERADOR SEGURO DE CONTRASEÑAS, utilizando diagramas funcionales y de arquitectura para representar el comportamiento y la estructura general del sistema. Estos diagramas nos permitirán identificar las interacciones entre el USUARIO y el SOFTWARE, así como los componentes principales que participan en su funcionamiento.

Investigación.

Diagramas de Funcionalidad

Los diagramas de funcionalidad, también llamados diagramas de bloques funcionales o ed comportamiento, ilustran que hace un sistema y como interactúan sus componentes para lograrlo. Son usados principalmente en ingeniería, diseño de procesos y software, los principales tipos se dividen según su enfoque.

1 Diagramas de funcionalidad de Software.

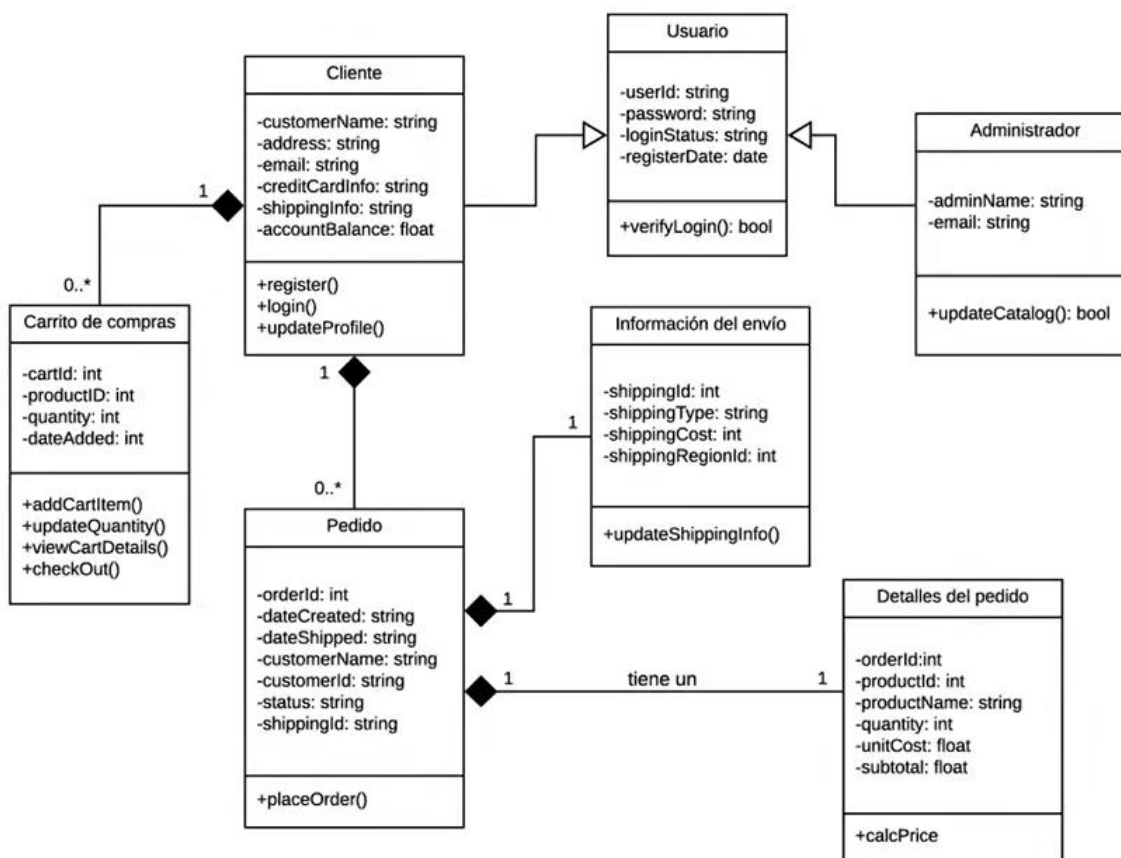
En el mundo del desarrollo de software, se utilizan los diagramas de Lenguaje unificado de Modelado (UML) para modelar la estructura y el comportamiento funcional.

- **Diagrama de Casos de Uso:** Muestran las funciones del sistema desde la perspectiva del USUARIO y que pueden lograr con el.
- **Diagrama de Actividades:** Representa el flujo secuencial, ramificado o concurrente de los procesos o casos de uso.
- **Diagrama de Secuencia:** Detalla como los objetos interactúan entre si a lo largo del tiempo para realizar una tarea específica.

- **Diagrama de Clases:** Los diagramas de clases son el tipo de diagrama UML más comúnmente usados, muestran la estructura estática de un sistema que incluye las clases, atributos y comportamientos, y las relaciones entra cada clase. Una clase esta representada por un rectángulo que contiene tres comportamientos apilados verticalmente: el comportamiento superior contiene l nombre de la clase y es obligatorio, pero los dos comportamientos inferiores muestran detalles sobre los atributos y las operaciones o comportamientos de la clase.

Figura 1.

Ejemplo de diagrama de clase.

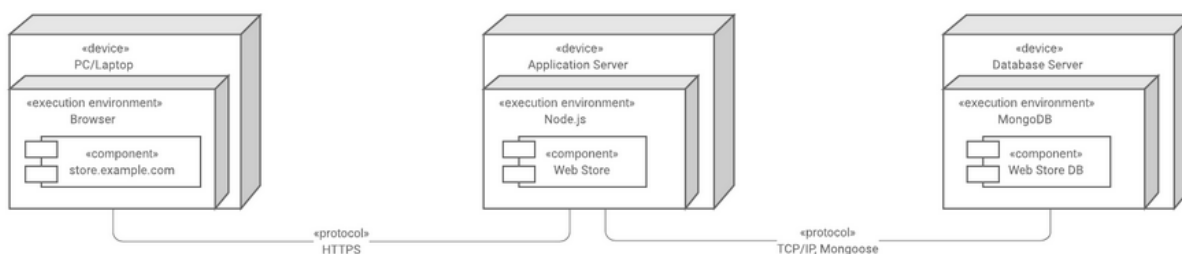


Nota. Adaptado de Lucid Chart, Diagramas de clase, (<https://www.lucidchart.com/blog/es/tipos-de-diagramas-uml>)

- **Diagramas de Componentes.** Un diagrama de componentes es esencialmente una versión más especializada del diagrama de clases: se aplican las mismas reglas de notación para ambos. Un diagrama de componentes descompone un sistema complejo en componentes más pequeños y visualiza la relación entre esos componentes.
- **Diagramas de Implementación.** Los diagramas de lanzamiento muestran cómo se implementa el software en los componentes de hardware de un sistema. Estos diagramas son muy útiles para los ingenieros de sistemas y suelen mostrar el rendimiento, la escalabilidad, la mantenibilidad y la portabilidad. Cuando los componentes de hardware se muestran en relación con los demás, es más fácil hacer un seguimiento de toda la malla de hardware y asegurarse de que todos los elementos se tienen en cuenta en una implementación.

Figura 2.

Ejemplo de diagrama de Lanzamiento.

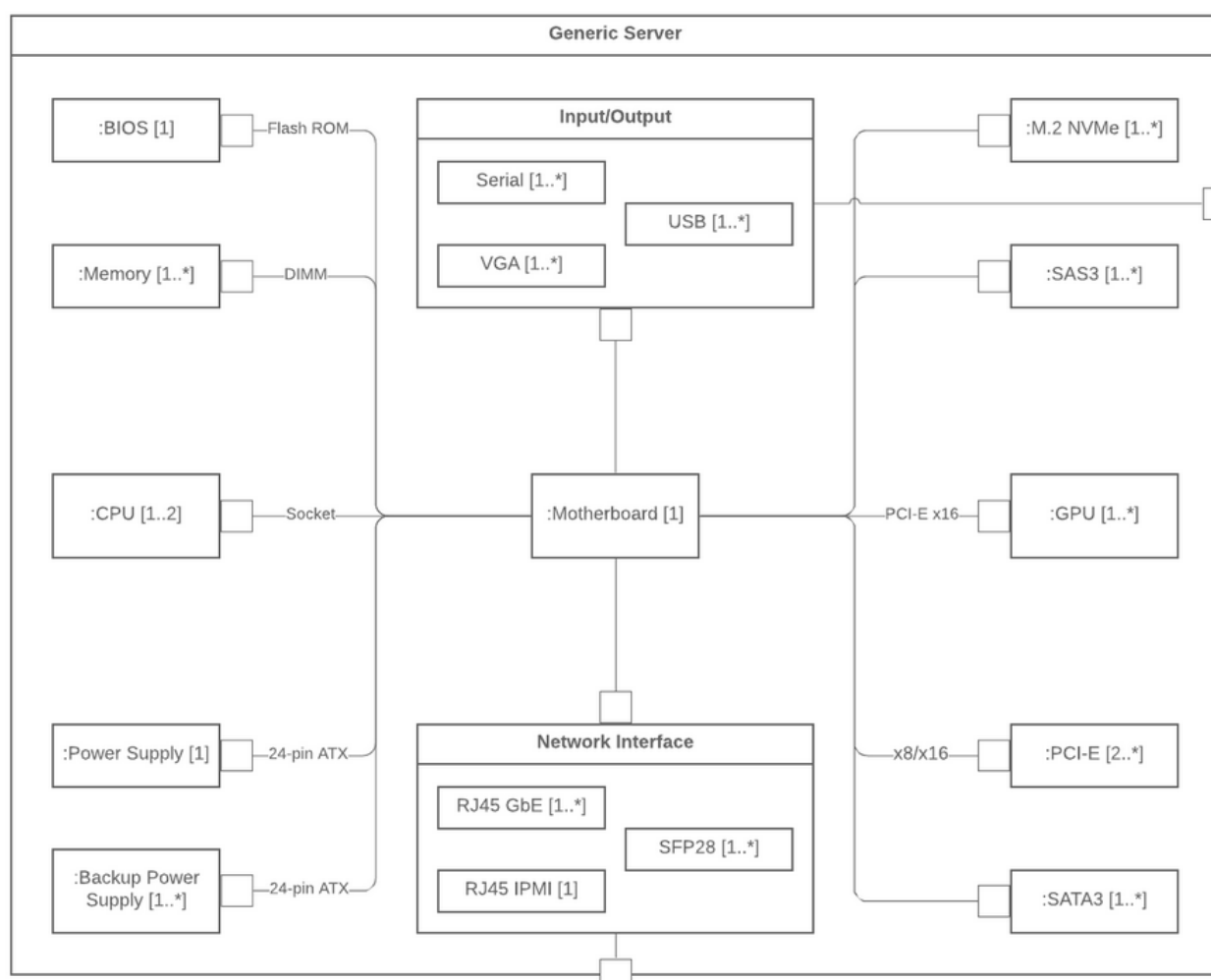


Nota. Adaptado de Lucid Chart, Diagramas de clase, (<https://www.lucidchart.com/blog/es/tipos-de-diagramas-uml>)

- **Diagramas de estructura compuesta.** Los diagramas de estructura compuesta son esencialmente los esquemas de la estructura interna de un clasificador, también muestra como el clasificador interactúa con el entorno a través de los puertos y son excelentes para mostrar los componentes interiores de cualquier hardware para comprender mejor su funcionamiento interno.

Figura 3.

Ejemplo de diagrama de estructura compuesta.

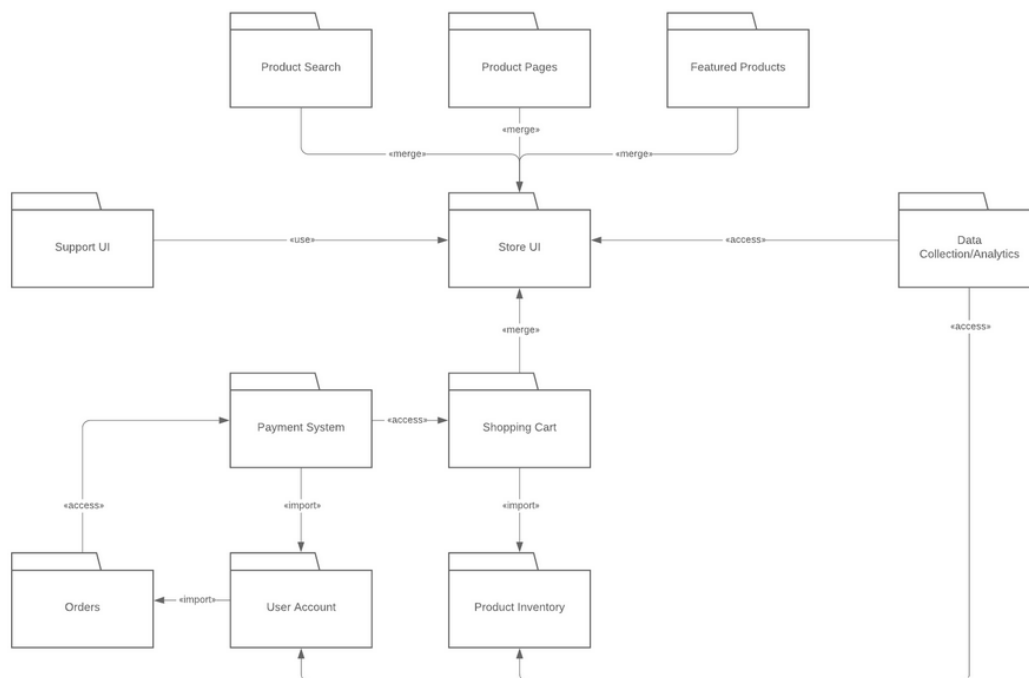


Nota. Adaptado de Lucid Chart, Diagramas de clase, (<https://www.lucidchart.com/blog/es/tipos-de-diagramas-uml>)

- **Diagrama de Objetos.** Muestran ejemplos de estructuras de datos en un momento específico. Podrías utilizar un diagrama de clases para mostrar una estructura y luego usar diagramas de objetos como casos de prueba para verificar la integridad de tu diagrama de clases. O bien, podrías crear un diagrama de objetos para descubrir información sobre los elementos de un modelo y sus vínculos.
- **Diagrama de paquetes.** Se utilizan para mostrar las dependencias entre los diferentes paquetes de un sistema. Un paquete, representado como una carpeta de archivos, organiza los elementos del modelo, como los casos de uso o las clases, en grupos.

Figura 4.

Ejemplo de Diagrama de paquetes.



Nota. Adaptado de Lucid Chart, Diagramas de clase, (<https://www.lucidchart.com/blog/es/tipos-de-diagramas-uml>)

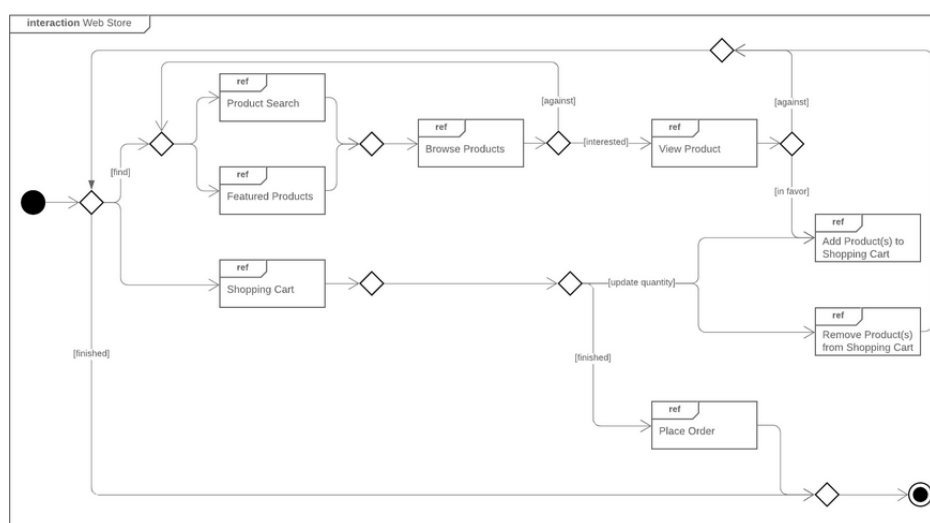
2 Diagramas UML de comportamiento.

Estos diagramas UML visualizan la forma en la que se comporta el sistema y cómo interactúa dentro de sí mismo, así como con los usuarios, otros sistemas y otras entidades.

- **Diagrama de temporización.** A menudo descrito como un diagrama de secuencia invertido, un diagrama de secuencia muestra cómo los objetos interactúan entre sí en un marco de tiempo determinado. Utiliza estos diagramas para ver cuánto tiempo lleva cada paso de un proceso y encontrar áreas de mejora.
- **Diagrama global de interacciones.** Este diagrama resume el flujo de control entre los nodos que interactúan. Incluyen nodos iniciales, nodos finales de flujo, nodos finales de actividad, nodos de decisión, nodos de fusión, nodos de bifurcación y nodos de unión.

Figura 5

Ejemplo de diagrama global de interacciones.

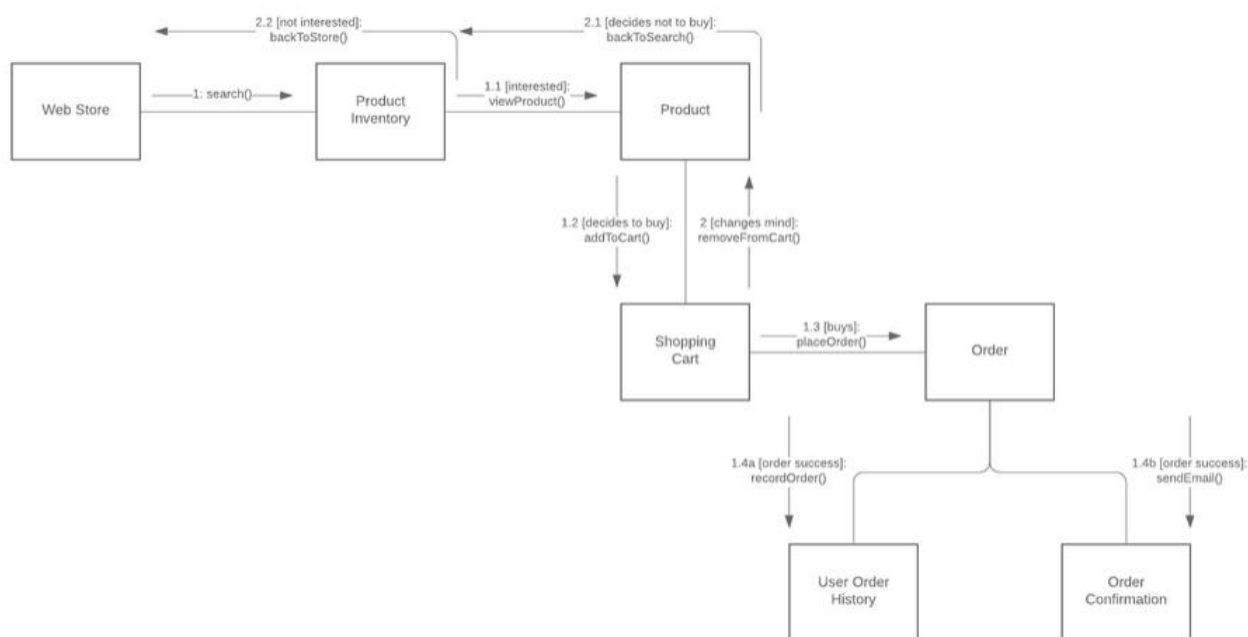


Nota. Adaptado de Lucid Chart, Diagramas de clase, (<https://www.lucidchart.com/blog/es/tipos-de-diagramas-uml>)

- **Diagrama de comunicación.** Conocidos anteriormente como diagramas de colaboración, muestran como se relacionan los objetos entre sí, modelan la forma en la que los objetos se asocian y conectan a través de mensajes dentro de un diseño arquitectónico de un sistema, también sirven para mostrar escenarios alternativos dentro de casos de uso u operaciones que requieren la colaboración de diferentes objetos e interacciones.

Figura 6.

Ejemplo de diagrama de comunicación.

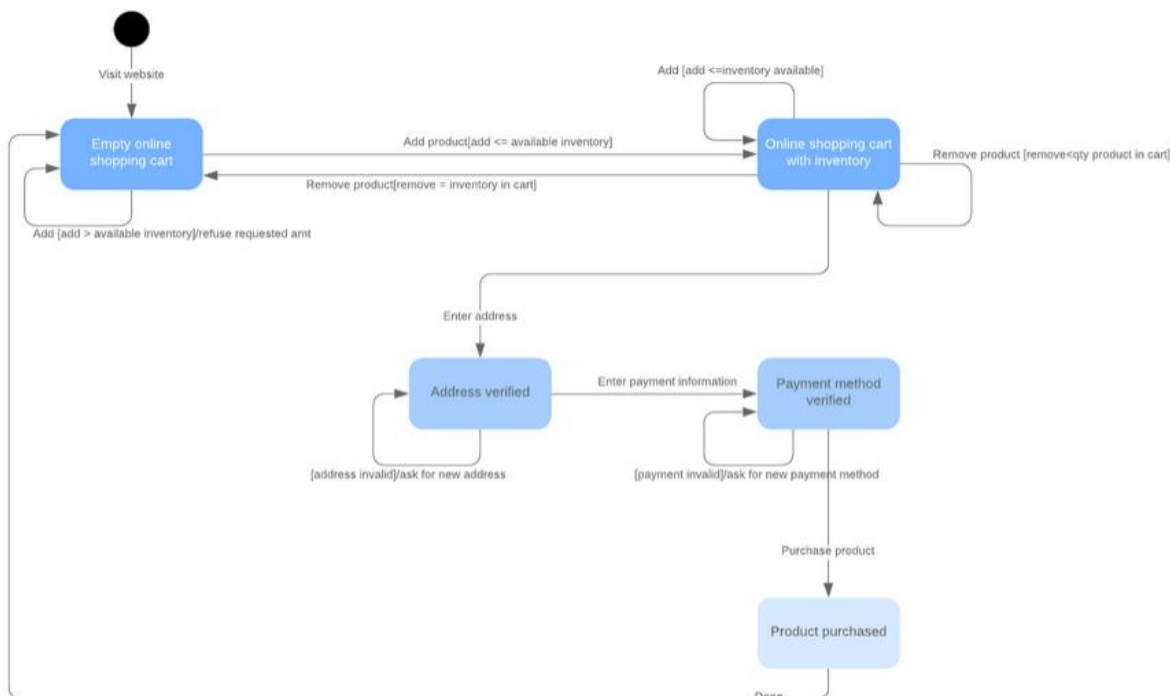


Nota. Adaptado de Lucid Chart, Diagramas de clase, (<https://www.lucidchart.com/blog/es/tipos-de-diagramas-uml>)

- **Diagrama de estados.** Simplemente representan estados y transiciones. Un estado se refiere a las diferentes combinaciones de información que puede tener un objeto y este diagrama UML puede visualizar todos los estados posibles y forma en que el objeto transita de un estado a otro.

Figura 7.

Ejemplo de diagrama de estados.



Nota. Adaptado de Lucid Chart, Diagramas de clase, (<https://www.lucidchart.com/blog/es/tipos-de-diagramas-uml>)

3 Diagramas de Flujo Funcional (Procesos).

Mapear las etapas de un proceso y las organizan por departamentos o roles.

- **Diagramas de Carriles (Swimlanes):** Dividen el flujo de trabajo en columnas o filas, cada carril representa la función o responsabilidad de un área o actor específico, dejando muy claro quien hace que.
- **Diagrama de flujo de datos (DFD):** Muestran como viaja la información a través de un sistema, indicando donde se procesa y donde se almacena.

4 Diagramas de bloques de funciones (FBD)

Comunes en la ingeniería de sistemas y automatización, describen la estructura y las funciones de elementos específicos.

- **Diagrama de Bloques Funcionales (BFD):** Desglosa un sistema grande en funciones mas pequeñas y conectadas, mostrando entradas, salidas y jerarquía funcional.

5 Diagramas de arquitectura

Un diagrama de arquitectura de software es una representación grafica que ilustra la estructura, los componentes, las relaciones y las interacciones de un sistema informático. Actua como un plano para el desarrollo, permitiendo visualizar la infraestructura y alinear los aspectos técnicos con los objetivos del negocio.

Casos de uso comunes.

- **Planificación y Diseño:** Ayuda a los equipos a definir y validar decisiones tecnológicas antes de escribir código.
- **Documentación Técnica:** Permite registrar cómo está construido el sistema para facilitar la incorporación de nuevos desarrolladores y el mantenimiento a largo plazo.
- **Análisis de Impacto:** Evalúa cómo afectaría un cambio (por ejemplo, actualizar una base de datos) a otras partes del sistema.
- **Auditoría y Seguridad:** Facilita la identificación de posibles vulnerabilidades al mapear el flujo de datos y las conexiones externas.
- **Comunicación:** Sirve como lenguaje común para explicar la estructura técnica tanto a programadores como a directivos y clientes.

Para crear estos diagramas, es habitual utilizar herramientas estándar del sector en la nube como Lucidchart o plataformas como Draw.io. Para flujos ágiles incrustados en código, también se emplea a menudo la sintaxis de Mermaid.js.

Selección y análisis del problema.

Problema seleccionado: Generador seguro de contraseñas.

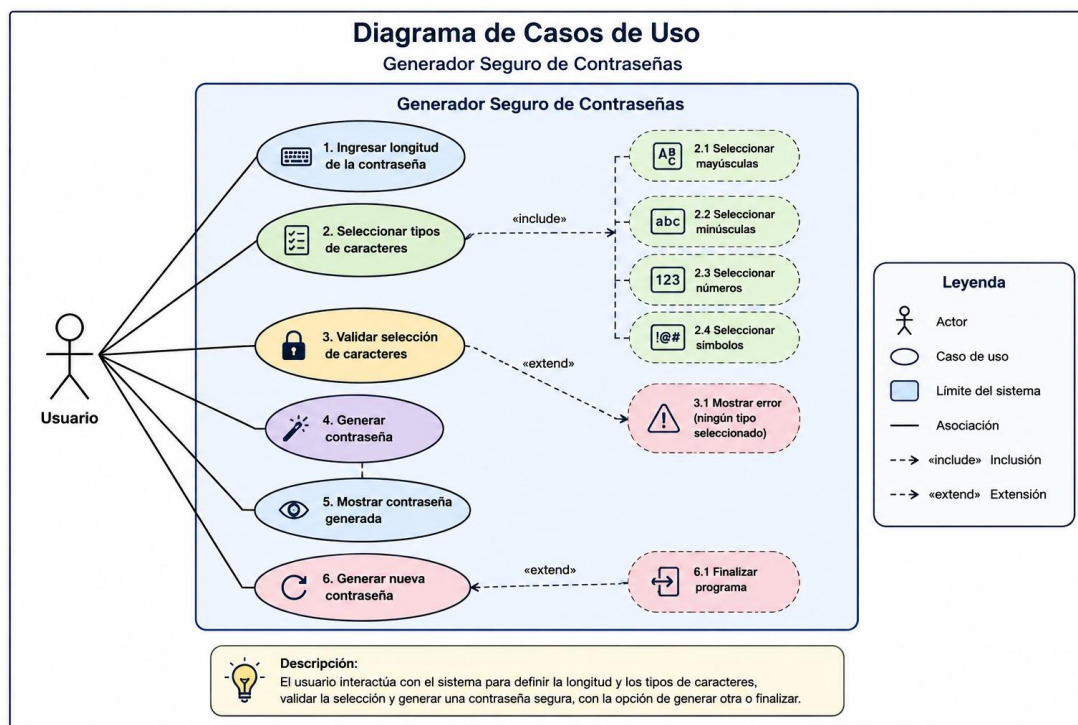
Descripción del problema.

Actualmente muchos usuarios utilizan contraseñas débiles que pueden ser vulnerables a ataques informáticos. Por esta razón se propone desarrollar una aplicación capaz de generar contraseñas seguras mediante la combinación aleatoria de letras mayúsculas, letras minúsculas, números y caracteres especiales.

Objetivo.

Desarrollar un sistema que permita generar contraseñas robustas y personalizadas según los criterios establecidos por el usuario.

Diseño Funcional - Diagrama UML de Casos de Uso.



Descripción de los Casos de Uso

Generar Contraseña.

Le va a permitir al actor en este caso los usuarios generar una contraseña aleatoria, pero más segura, de esta manera evitaremos las vulnerabilidades con contraseñas deficientes.

Configurar Longitud.

Permitirá al sistema definir la cantidad de caracteres de la contraseña, las recomendaciones actuales de ciberseguridad (como las NIST) priorizan la longitud de la contraseña sobre la dificultad, para una configuración Robusta se recomienda una longitud mínima de 12 a 15 caracteres, siendo el ideal 16 caracteres o más.

Seleccionar Caracteres.

Permite al software decidir si se incluirán caracteres de texto, numero, caracteres o símbolos especiales, sin embargo, según las recomendaciones actuales de ciberseguridad, ya no es obligatorio mezclar caracteres como Mayúsculas, minúsculas, números y caracteres especiales cuando la contraseña es lo suficientemente larga.

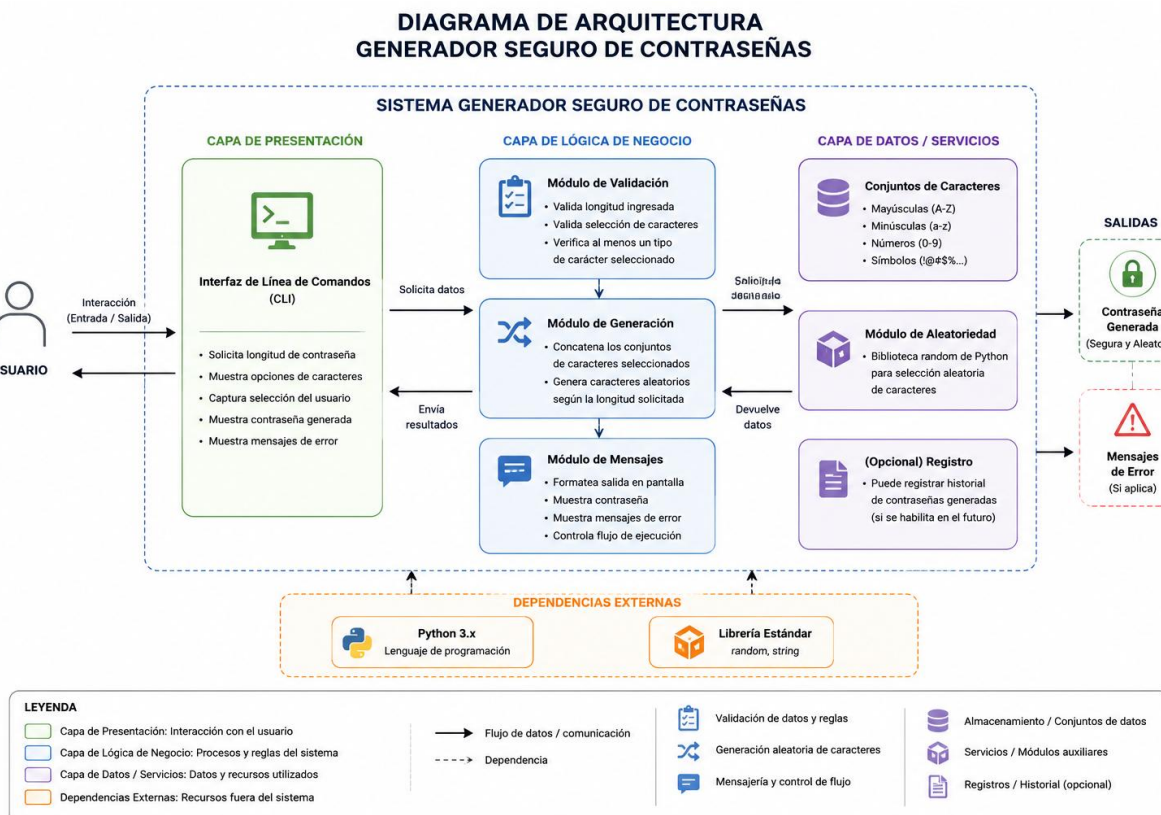
Visualizar Contraseña.

Mostrará el resultado generado desde el software para el Usuario.

Copiar Contraseña.

Permitirá al usuario copiar la contraseña generada para su uso posterior.

Diseño de Arquitectura



Conclusiones

1. El diseño previo del software permite comprender las necesidades del sistema antes de iniciar la programación.
2. Los diagramas funcionales facilitan la identificación de las acciones que realizará el usuario dentro de la aplicación.
3. El diagrama de arquitectura permite visualizar los componentes principales del sistema y su interacción para generar contraseñas seguras.

Recomendaciones

1. Realizar siempre un análisis previo antes de comenzar el desarrollo de software.
2. Utilizar diagramas UML para mejorar la comprensión del sistema.
3. Mantener una arquitectura organizada que facilite futuras mejoras y mantenimiento de la aplicación.

REFERENCIAS

- Booch, G., Rumbaugh, J., & Jacobson, I. (2005). The unified modeling language user guide (2nd ed.). Addison-Wesley Professional.
- Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design patterns: Elements of reusable object-oriented software. Addison-Wesley.
- Larman, C. (2004). Applying UML and patterns: An introduction to object-oriented analysis and design and iterative development (3rd ed.). Prentice Hall.
- Pressman, R. S., & Maxim, B. R. (2020). Software engineering: A practitioner's approach (9th ed.). McGraw-Hill Education.
- Sommerville, I. (2016). Software engineering (10th ed.). Pearson Education.
- Object Management Group. (2023). Unified Modeling Language (UML).
<https://www.omg.org/spec/UML>
- Microsoft. (2024). Security best practices for password management.
<https://learn.microsoft.com>
- Oracle. (2024). Java secure coding guidelines. <https://www.oracle.com/java>
- LucidChart (2019), <https://www.lucidchart.com/blog/es/tipos-de-diagramas-uml>